

컴퓨터비전 _인공지능 (AI)개론

8. 이진 분류

- 문제 정의하기
- 정확도(Accuracy)
- 훈련 데이터와 검증 데이터 분할
- 시그모이드 함수
- 교차 엔트로피 함수
- 데이터 준비
- 모델 정의
- 경사 하강법
- 결과 확인

우리가 지금까지 배운 '회귀' 모델은 연속된 숫자를 예측하는 '값 맞추기' 게임과 같았습니다.

하지만 세상의 많은 문제는 '값 맞추기'가 아니라 '편 가르기'에 가깝습니다.

메일이 스팸인지 아닌지, 사진 속 동물이 고양이인지 강아지인지, 신용카드 거래가 정상인지 사기인지 판단하는 것처럼 말이죠.

이처럼 주어진 데이터를 **두 그룹 중 하나로 나누는 작업을 이진 분류** 라고 합니다.

데이터 구성

세 종류의 붓꽃에 대해

'꽃잎'과 '꽃받침'의 길이와 폭을 측정한
데이터입니다.

이진 분류 개념에 집중하기 위해

두 종류의 붓꽃 데이터만 사용하고,

시각화를 위해 두 가지 특성만 활용합니다.

데이터 특징

- 총 150개의 데이터 포인트
- 4개의 특성 (꽃받침 길이/폭, 꽃잎 길이/폭)
- 3개의 붓꽃 품종 분류
- 이진 분류를 위해 2개 품종만 선택



선형 회귀 모델과 비교했을 때,

nn.Sigmoid라는 시그모이드 함수가 마지막에 추가 된 점이 가장 큰 차이점입니다.

이 구조를 이진 로지스틱 회귀 모델 이라고 부릅니다.

회귀 모델의 평가

손실(오차)이 얼마나 작은지가 중요했지만, 그 값이 '얼마나' 작아야 좋은 모델인지 직관적으로 알기 어려웠습니다.

분류 모델의 평가

성적을 매기기가 훨씬 쉽습니다. 예측이 정답과 '맞았는지' 혹은 '틀렸는지'만 판단하면 되니까요.

정확도 = (정답 건수) / (전체 건수)

예시: 10개 데이터 중 9개를 맞혔다면 정확도는 $9/10 = 0.9$ (90%)

어떤 학생이 족보에 있는 문제만 달달 외웠다고 상상해봅시다.
족집게 시험에서는 100점을 맞겠지만,
처음 보는 문제가 나오는 실제 시험에서는 좋은 성적을 기대하기 어렵겠죠.

딥러닝 모델도 마찬가지입니다.
학습에 사용한 데이터에만 너무 익숙해져서,
새로운 데이터에 대한 예측 능력이 떨어지는 현상을 **과학습(Overfitting)**이라고
합니다.

데이터 분할

훈련 데이터: 모델이 학습하는 데 사용 (연습 문제)

검증 데이터: 성능 평가에만 사용 (모의고사)



함수 특성

수학식: $f(x) = 1/(1+\exp(-x))$.

어떤 입력 값이든

항상 0과 1 사이의 값을 출력합니다.

입력 값이 0일 때

출력 값은 정확히 0.5가 됩니다.



확률 변환

선형 함수의 출력값은

범위 제한이 없는 숫자입니다.

하지만 분류 문제에서는

"이 데이터가 정답일 확률은 70%"과

같은 확률 값이 필요합니다.



예측 기준

모델의 최종 출력값을

'정답이 1일 확률'로 해석할 수
있습니다.

출력이 0.5보다 크면 1로,

작으면 0으로 예측할 수 있습니다.

회귀 모델에서는 평균 제곱 오차(MSE)를 손실 함수로 사용했습니다. 분류 모델에서는 **교차 엔트로피 함수**를 사용합니다.

학생 A

"정답은 '사과'인 것 같아요..." → 틀렸을 때

학생 B

"정답은 100% '바나나'입니다!" → 틀렸을 때

누구에게 더 큰 벌점을 주어야 할까요? 당연히 강한 확신을 가지고 틀린 학생 B입니다.

교차 엔트로피는 모델이 높은 확신을 가지고 틀렸을 때 훨씬 더 큰 페널티를 부여합니다.

01

데이터 불러오기

사이킷런의 `load_iris()`를 사용하여 붓꽃 데이터셋을 불러옵니다.
원본 데이터는 150개 데이터, 4개 특성을 가집니다.

03

데이터 분할

`train_test_split` 함수를 사용하여
훈련용 70개, 검증용 30개로 데이터를 나눕니다.

02

데이터 추출

이진 분류를 위해 두 종류의 꽃 데이터(100개)와
두 개의 특성(꽃받침 길이/너비)만 사용합니다.

04

텐서 변환

NumPy 배열을 파이토치 텐서로 변환하고,
정답 텐서의 shape을 (N, 1) 형태로 조정합니다.

```
# 모델 정의
# 2입력 1출력 로지스틱 회귀 모델
class Net(nn.Module):
    def __init__(self, n_input, n_output):
        super().__init__()
        self.l1 = nn.Linear(n_input, n_output)

        # 초깃값을 모두 1로 함
        self.l1.weight.data.fill_(1.0)
        self.l1.bias.data.fill_(1.0)

# 예측 함수 정의
def forward(self, x):
    # 입력 값과 행렬 곱을 계산
    x1 = self.l1(x)
    return x1
```

__init__ 메서드

모델이 사용할 계층을 정의합니다. 2개 입력을 받아 1개 출력을 내는 nn.Linear 계층과 확률로 변환할 nn.Sigmoid 계층을 정의합니다.

forward 메서드

데이터가 계층들을 통과하는 순서를 정의합니다. 입력이 선형 함수를 거친 후 시그모이드 함수를 통과하여 최종 결과가 나옵니다.

1

경사값 초기화

`optimizer.zero_grad()`로 이전 반복의 경사값을 초기화합니다.

2

순전파

모델에 입력 데이터를 넣어 예측값을 계산합니다.

3

손실 계산

예측값과 정답을 비교하여 교차 엔트로피 손실을 계산합니다.

4

역전파

`loss.backward()`로 각 파라미터에 대한 경사를 계산합니다.

5

파라미터 업데이트

`optimizer.step()`으로 계산된 경사를 바탕으로 파라미터를 수정합니다.

50% 96.7%

초기 정확도

최종 정확도

학습 시작 시 무작위 예측 수준

검증 데이터에서 달성한 최종 성능

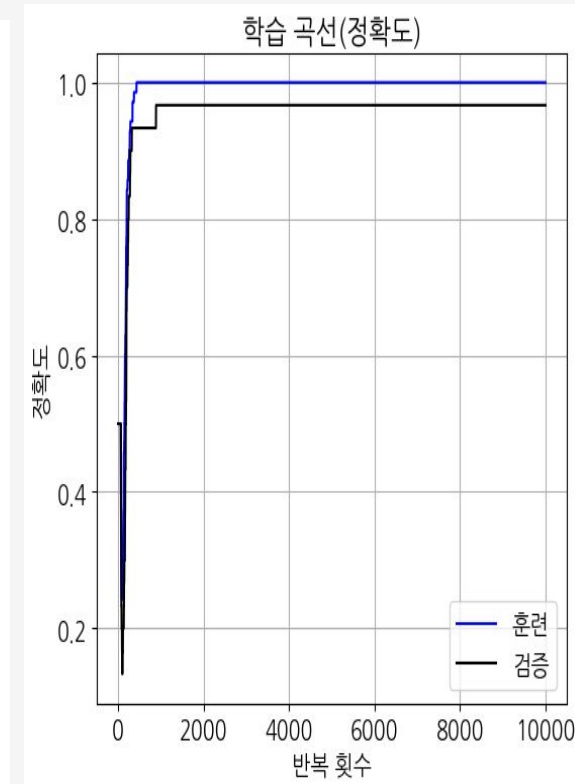
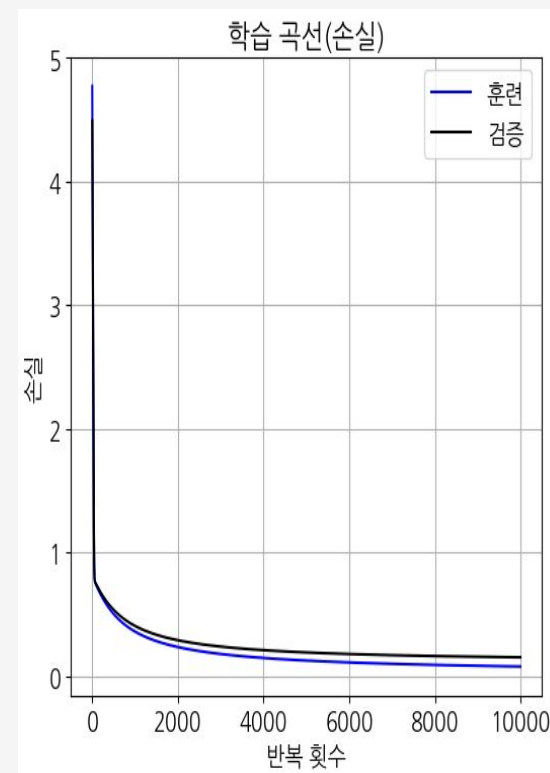
학습 곡선을 보면

훈련 데이터와 검증 데이터의 손실과 정확도가

거의 동일한 패턴으로 움직입니다.

이는 모델이 과적합된 학습 없이

일반화된 예측을 잘 수행하고 있음을 의미합니다.



로지스틱 회귀 모델은 데이터 공간을 가르는 하나의 직선을 학습합니다. 이를 **결정 경계**라고 합니다.

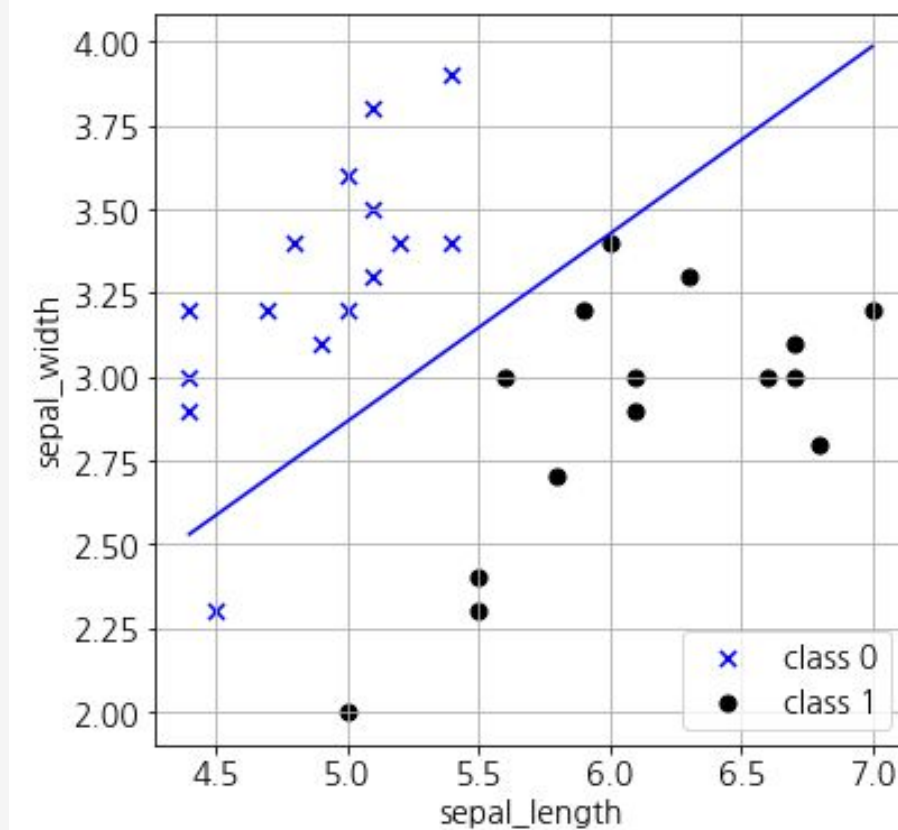
학습된 파라미터

BIAS = [0.3386], WEIGHT = [[2.97 -5.3]]

이 값들이 결정 경계선의 위치와 기울기를 결정합니다.

분류 성능

결정 경계선이 두 그룹(setosa, versicolor)을 거의 완벽하게 나누는 것을 확인할 수 있습니다.



기존 방식 (BCELoss)

- 모델: `nn.Linear` → `nn.Sigmoid`
- 손실 함수: `nn.BCELoss()`
- 예측 기준: 0.5

두 번에 나눠 처리하는 방식

권장 방식 (BCEWithLogitsLoss)

- 모델: `nn.Linear`만 사용
- 손실 함수: `nn.BCEWithLogitsLoss()`
- 예측 기준: 0.0

내부에 Sigmoid 기능 포함

❏ **수치적 안정성:** BCEWithLogitsLoss는 시그모이드와 교차 엔트로피 계산을 하나로 합쳐 수학적으로 더 안정적인 방식으로 처리합니다.



이진 분류

주어진 데이터를 두 그룹 중 하나로 나누는 작업
스팸 메일 분류, 의료 진단 등에 활용



시그모이드 함수

선형 함수의 출력을 0과 1 사이의 확률값으로 변환하는 함수



정확도

전체 문제 중 몇 개를 맞혔는지를 나타내는 직관적인 성능 지표



교차 엔트로피

분류 모델의 손실 함수로,
확신을 가지고 틀렸을 때 더 큰 페널티를 부여